Deep Reinforcement Learning Approach to Knapsack Problem

Sadra Moosavi Lar, Tijn van der Zwan, Amirhossein Setayeshgar, Tariq Oduber
Eindhoven University of Technology (TU/e), Netherlands
{s.s.moosavi.lar, t.v.d.zwan, a.setayeshgar, t.j.oduber}@student.tue.nl

***Abstract*—This paper studies the bounded knapsack problem as a reinforcement learning task using Deep Q-Network (DQN), Proximal Policy Optimization (PPO), and Maskable PPO. The environment is implemented in Gymnasium and adapted from OR-GYM. We evaluate the agents over multiple random seeds, compare their performance, and analyze the effect of hyperparameter tuning. The results show that action masking substantially improves learning by preventing invalid actions, leading to higher rewards and more stable performance.**

***Index Terms*—Reinforcement Learning, DQN, PPO, Knapsack Problem, Action Masking**

I. Part 1: The environment

The bounded knapsack problem is a combinatorial optimization problem in which an agent must select items with different weights and values to maximize the total value packed in a knapsack without exceeding its capacity. Unlike the unbounded case, each item can only be chosen a limited number of times. The problem can be written as

$$\max \sum_{i=1}^{n} p_i x_i \tag{1}$$

subject to

$$\sum_{i=1}^{n} w_i x_i \leq W, \tag{2}$$

$$0 \leq x_i \leq c_i, \quad x_i \in \mathbb{Z}, \tag{3}$$

where p_i is the value of item i, w_i is its weight, c_i is its maximum available quantity, and x_i is the number of times item i is selected.

This problem is modeled as a Markov Decision Process (MDP) using the `BoundedKnapsackEnv`, which is adapted from OR-GYM and implemented in Gymnasium. The environment used in this assignment contains 200 items and a maximum knapsack weight of 200. At each step, the agent observes the current state of the problem and chooses one item to pack. The state includes the item weights, item values, item limits, the current knapsack load, and the maximum capacity. Since the next outcome depends only on the current state and chosen action, the problem satisfies the Markov property.

The MDP can be defined through its main components. The **state** s_t contains all relevant information for decision-making at time step t: the item weights, item values, remaining item limits, knapsack capacity, and current packed weight. The **action space** is discrete, where each action $a_t \in \{0, 1, \ldots, n-1\}$

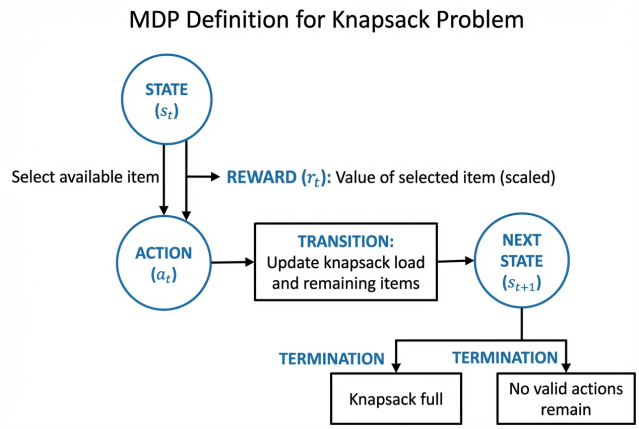

Fig. 1. Illustration of the bounded knapsack problem as a Markov Decision Process. At each step, the agent observes the current state, selects an item, receives a reward, and transitions to a new state until the episode terminates.

corresponds to selecting one item. The **transition function** updates the knapsack after an action: if the selected item is still available and fits within the remaining capacity, its weight is added to the knapsack and its remaining limit is reduced by one. Otherwise, the episode ends immediately. The **reward** equals the value of the chosen item if the action is valid; in the implementation this reward is scaled by 0.01 to improve training stability. Invalid actions return zero reward. Finally, the **episode terminates** when the knapsack is exactly full or when the agent chooses an item that no longer fits or is no longer available.

The interaction between the reinforcement learning agent and the environment is sequential. At the beginning of each episode, the environment is reset, which empties the knapsack and restores the item limits. The agent then repeatedly observes the current state, selects an action according to its policy, and receives the next state and reward from the environment. Through many episodes, the agent learns which sequence of item selections leads to the highest cumulative reward. In this way, the bounded knapsack problem becomes a sequential decision-making task that can be solved using reinforcement learning.

II. Part 2: Training and Tuning DQN and PPO

In this part, we trained and compared Deep Q-Network (DQN) and Proximal Policy Optimization (PPO) on the bounded knapsack environment. DQN is a value-based method

that learns an approximation of the action-value function $Q(s, a)$ and updates its neural network by minimizing the temporal-difference loss

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]. \quad (4)$$

Here, θ denotes the online network parameters and θ^- the target network parameters. DQN updates its weights from mini-batches sampled from a replay buffer, so updates happen repeatedly during training after enough transitions have been collected. PPO, in contrast, is a policy-gradient method that directly updates the policy using the clipped objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \quad (5)$$

where $r_t(\theta)$ is the ratio between new and old policy probabilities and $\hat{A}_t$ is the estimated advantage. PPO first collects a rollout of interactions and then performs several gradient updates on that batch. In short, DQN learns state-action values from replayed transitions, while PPO updates a stochastic policy from recent trajectories.

Both methods were implemented using Stable-Baselines3 and evaluated with the same three seeds: 21, 31, and 41. The final experiments used 50,000 training steps. Evaluation was performed every 2,000 steps over 5 episodes, and results were aggregated across seeds using the mean and standard deviation.

Table I shows the default performance. Figure 2 shows the training curves of DQN and PPO with default hyperparameters.

DQN achieves a higher mean reward but exhibits high variance across seeds, indicating unstable learning. In contrast, PPO shows lower average performance but more stable behavior.

TABLE I
FINAL PERFORMANCE WITH DEFAULT HYPERPARAMETERS

Method	Mean reward	Std. dev.
DQN	410.67	285.20
PPO	62.33	66.26

For tuning, we selected two shared hyperparameters and two method-specific hyperparameters for each algorithm. The shared hyperparameters were the learning rate and the discount factor γ, because both directly affect how quickly the agent learns and how strongly it values future reward. For DQN, we additionally tuned the replay buffer size and the initial exploration rate, since these influence the diversity of stored experience and early exploration behaviour. For PPO, we additionally tuned `n_steps` and the clipping range, because these control the rollout size before each update and the stability of policy updates. The selected hyperparameters and their final chosen values are summarized in Tables II and III.

The tuning trends changed compared with the shorter training runs. For DQN, the largest tested learning rate (0.0001) again gave the best mean reward, while $\gamma = 0.97$ outperformed

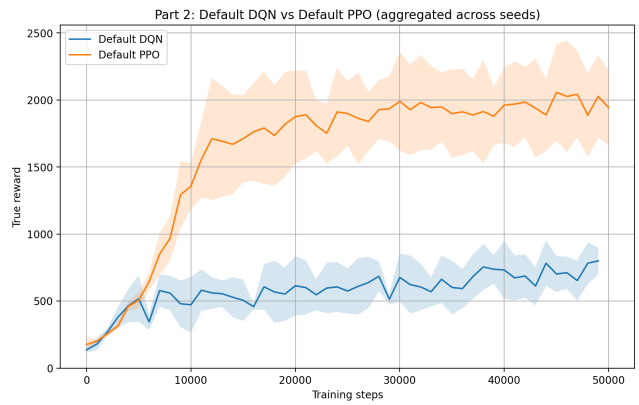

Fig. 2. Aggregated training performance of DQN and PPO with default hyperparameters.

TABLE II
SELECTED DQN HYPERPARAMETERS, MOTIVATION, AND CHOSEN VALUES

Hyperparameter	Reason for selection	Best value
Learning rate	Controls update size and convergence speed; too small slows learning, too large may cause instability.	0.0001
γ	Determines how much future reward is considered; important for balancing immediate and long-term value.	0.97
Buffer size	Affects diversity of replayed transitions and learning stability in DQN.	10000
Exploration initial epsilon	Controls how much the agent explores at the start of training before exploiting learned values.	1.0

TABLE III
SELECTED PPO HYPERPARAMETERS, MOTIVATION, AND CHOSEN VALUES

Hyperparameter	Reason for selection	Best value
Learning rate	Controls update size and affects whether policy learning is stable or too aggressive.	0.00001
γ	Determines the importance of future reward in advantage estimation and policy improvement.	0.9
`n_steps`	Controls how many interactions are collected before each policy update; affects sample quality and update frequency.	512
Clip range	Limits how far the policy may change in one update and is central to PPO stability.	0.1

the smaller discount factors. A larger replay buffer also performed best, with buffer size = 10000 clearly exceeding the smaller tested values. For PPO, the best learning rate became 0.00001, while $\gamma = 0.9$, `n_steps`= 512, and clip range = 0.1 gave the strongest average results among the tested candidates. These results suggest that under a longer training budget, PPO benefited from more conservative updates, whereas DQN benefited from placing more emphasis on future reward and storing more experience.

TABLE IV
FINAL PERFORMANCE AFTER TUNING

Method	Mean reward	Std. dev.
Tuned DQN	365.33	185.35
Tuned PPO	238.67	143.12

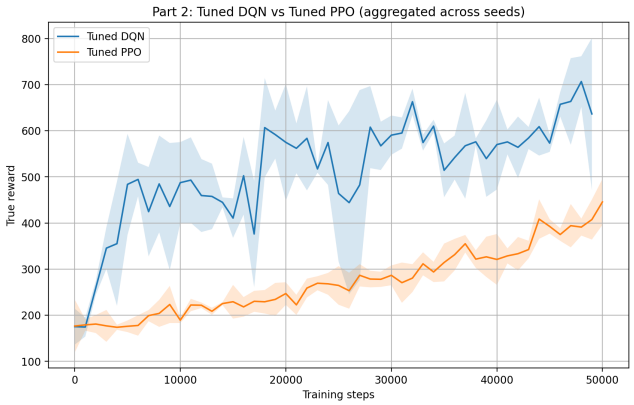

Fig. 3. Aggregated training performance of DQN and PPO after hyperparameter tuning.

Overall, DQN achieved the strongest default performance, suggesting that value-based learning with replay remained effective in this environment. However, its large standard deviation shows that it was still sensitive to randomness across seeds. PPO performed poorly with default hyperparameters, but its tuned version improved substantially, reaching 238.67 mean reward compared with only 62.33 for the default setting. In contrast, DQN tuning produced only a moderate decrease from the already strong default result. This indicates that, under 50,000 training steps, hyperparameter tuning was especially important for PPO, while DQN was already relatively competitive with its default configuration.

All reported results are averaged over three random seeds (21, 31, 41), and the shaded regions in the plots represent the standard deviation across these seeds.

III. PART 3: TRAINING WITH ACTION MASKING

In this part, we extended PPO by enabling action masking and training a MaskablePPO agent from SB3 Contrib on the bounded knapsack environment. Action masking prevents the agent from selecting invalid actions, such as items that no longer fit in the remaining capacity or items whose limit has already been exhausted. Instead of learning to avoid these actions only through trial and error, the agent is explicitly guided toward feasible choices. This is especially useful in the bounded knapsack problem, where many actions become invalid as the episode progresses.

To ensure a fair comparison with Part 2, we used the same three random seeds: 21, 31, and 41, and trained for 50,000 timesteps. We first trained a baseline MaskablePPO agent using the best PPO configuration from Part 2, namely learning rate = 0.00001, `n_steps`= 512, batch size = 32, `n_epochs`= 5, clip range = 0.1, and $\gamma = 0.9$. This masked baseline achieved a final mean reward of 575.33 with a standard deviation of 212.74, which is much higher than the tuned PPO result from Part 2 (238.67 $\pm$ 143.12). This shows that action masking substantially improved average performance, although the variability across seeds remained relatively high.

After establishing the masked baseline, we explored additional hyperparameter values to determine whether performance could be improved further. Since the PPO parameters from Part 2 already provided a strong starting point, we focused on a small number of impactful settings: learning rate, `n_steps`, and entropy coefficient (`ent_coef`). The learning rate was retained because it controls the size of policy updates, `n_steps` was included because action masking changes the structure of the collected rollouts, and `ent_coef` was added because entropy regularization can affect the exploration-exploitation trade-off even when invalid actions are removed. The selected hyperparameters and chosen values are summarized in Table V.

TABLE V
SELECTED MASKABLEPPO HYPERPARAMETERS, MOTIVATION, AND CHOSEN VALUES

Hyperparameter	Reason for selection	Best value
Learning rate	Controls update size and affects whether policy learning is stable or too aggressive.	0.0001
`n_steps`	Controls how many interactions are collected before each update and may behave differently under masking.	128
`ent_coef`	Encourages exploration and helps control how deterministic the masked policy becomes during training.	0.0

The tuning results showed that the best learning rate was 0.0001, while shorter rollouts again performed best for the masked setting: `n_steps`= 128 gave a higher mean reward than 256 or 512. For entropy regularization, the best result was obtained with `ent_coef`= 0.0, indicating that additional entropy encouragement did not help in this case. The resulting tuned MaskablePPO configuration therefore used learning rate = 0.0001, `n_steps`= 128, batch size = 32, `n_epochs`= 5, clip range = 0.1, $\gamma = 0.9$, and `ent_coef`= 0.0. However, the final tuned run reached a mean reward of 301.00 with a standard deviation of 81.80, which was substantially lower than the masked baseline.

TABLE VI
COMPARISON OF PPO AND MASKABLEPPO PERFORMANCE

Method	Mean reward	Std. dev.
Tuned PPO (Part 2)	238.67	143.12
Masked PPO baseline	575.33	212.74
Tuned Masked PPO	301.00	81.80

Table VI summarizes the final performance of PPO and MaskablePPO variants.

Figure 4 shows the training curves of tuned PPO and MaskablePPO variants. The masked methods consistently achieve higher rewards across training, demonstrating that removing invalid actions simplifies the learning problem and leads to more effective policy updates.

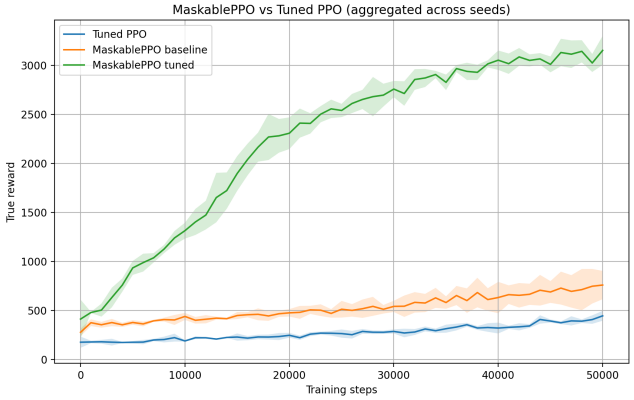

Fig. 4. Comparison between tuned PPO (Part 2) and MaskablePPO variants. Masked PPO clearly achieves higher rewards, showing the effectiveness of invalid action masking.

Overall, the results show that action masking was the most important improvement. Even without further tuning, MaskablePPO clearly outperformed the tuned PPO model from Part 2, suggesting that explicitly removing invalid actions makes the learning problem much easier. However, additional tuning did not improve the final result and instead reduced the mean reward, even though it also reduced the standard deviation. This indicates that, for this environment and training budget, the main gain came from masking itself rather than from further hyperparameter optimization.

IV. FIGURES

This section presents the training curves for the evaluated agents. Together, these figures illustrate both the average learning behaviour and the variability between runs.

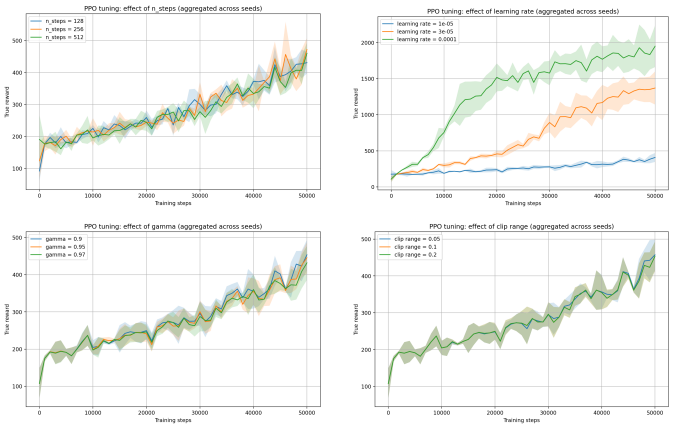

Fig. 5. Hyperparameter sensitivity of PPO aggregated across the three seeds. Top-left: effect of `n_steps`. Top-right: effect of learning rate. Bottom-left: effect of γ. Bottom-right: effect of clip range. In all plots, the solid lines show the mean true reward and the shaded regions indicate the standard deviation across seeds.

Fig. 6. Hyperparameter sensitivity of DQN aggregated across the three seeds. Top-left: effect of learning rate. Top-right: effect of γ. Bottom-left: effect of exploration initial epsilon. Bottom-right: effect of buffer size. In all plots, the solid lines show the mean true reward and the shaded regions indicate the standard deviation across seeds.

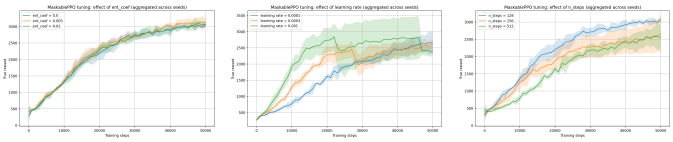

Fig. 7. Hyperparameter sensitivity of MaskablePPO aggregated across the three seeds. Left: effect of `ent_coef`. Center: effect of learning rate. Right: effect of `n_steps`. In all plots, the solid lines show the mean true reward and the shaded regions indicate the standard deviation across seeds.